

Siemens S7

Background

Fieldbus là một loại mạng công nghiệp, dùng để kết nối PLC với các thiết bị hiện trường (sensor, actuator), kết nối các PLC với nhau hoặc kết nối PLC với các thiết bị công nghiệp khác. Mỗi chuẩn fieldbus quy định:

- loại cáp
- cách truyền tín hiệu
- giao thức truyền dữ liệu
- cách tổ chức mạng

Một số loại Fieldbus phổ biến:

Tên Fieldbus	Cáp	Topology	Giao thức truyền thông
PROFIBUS	cáp RS-485	bus	PROFIBUS
Modbus RTU	cáp RS-485	bus	Modbus
CAN bus	twisted pair	bus	CANopen, DeviceNet

Ethernet công nghiệp là một loại mạng công nghiệp khác sử dụng cáp Ethernet tiêu chuẩn. Các giao thức phổ biến chạy trên Ethernet công nghiệp bao gồm EtherNet/IP, Modbus TCP, **PROFINET** và **Siemens S7**.

Ưu điểm của mạng Ethernet công nghiệp là:

- Tốc độ cao, tương thích tốt hơn với các thiết bị IT và dễ gỡ lỗi.
- Chi phí thấp hơn so với các loại fieldbus truyền thống do sử dụng hạ tầng Ethernet tiêu chuẩn và không yêu cầu các bộ chuyển đổi chuyên dụng.

Các PLC của Siemens có thể giao tiếp qua cáp Ethernet vật lý bằng một trong hai giao thức sau [\[1\]](#):

- **Open TCP/IP**: triển khai tiêu chuẩn của TCP/IP, nghĩa là PLC sử dụng TCP/IP thông thường. Khi đó, ta có thể xây dựng chương trình như khi lập trình mạng thông thường bằng socket, TCP hoặc UDP. Tuy nhiên, giao thức này chỉ truyền dữ liệu dưới dạng luồng (stream-based), trong khi các lệnh điều khiển của Siemens cần có độ dài cố định (Data Block). Vì vậy, trong chương trình PLC, lập trình viên phải sử dụng FC (Function Code) và FB (Function Block) để tự đóng gói thông điệp (packetize) từ luồng dữ liệu thành các khối.
- **S7, S7comm hoặc S7 Communication, và bản nâng cấp sau này là S7comm Plus**: Đây là giao thức độc quyền của Siemens, xuất hiện lần đầu vào năm 1994 cùng với sự ra đời của các dòng sản phẩm SIMATIC S7-200, S7-300 và S7-400. Trong kiến trúc mạng công nghiệp, giao thức S7 hoạt động theo mô hình request-response.

Một số tài liệu sử dụng các tên gọi khác như mô hình client-server (khi S7 chạy trên nền Ethernet hiện đại, với hạ tầng mạng có switch hỗ trợ kết nối full-duplex) hoặc master-slave (khi S7 chạy trên mạng fieldbus cũ như PROFIBUS, với hạ tầng mạng chỉ gồm một bus dùng chung). Dù gọi là gì thì về bản chất, S7 vẫn tuân theo mô hình giao tiếp request-response, tức là một bên gửi yêu cầu và bên kia trả lời [\[4\]](#).

	Profinet				implemented S7 Protocol			
Application Layer	Profinet IO RT / IRT	Profinet IO	Profinet CBA	Profinet CBA RT	S7 Comm Plus	S7 Comm	S7 Comm	
Presentation Layer								
Session Layer		RPC	BCOM					
Transport Layer		UDP	TCP		ISO Transport Protocol RFC 905 (Class 0)			
					ISO on TCP RFC 1006	TCP	ISO Transport Protocol RFC 905 (Class 4)	
Network Layer	IP		IP					
Data Link Layer	Industrial Ethernet							
Physical Layer								

Cách đọc địa chỉ trong Siemens S7

[Loại vùng nhớ] [Kích thước dữ liệu] [Địa chỉ byte].[Địa chỉ bit]

offset tính theo bit từ đầu Byte này

offset tính theo Byte từ đầu vùng nhớ này

X(bit) B(byte) W(word) D(dword) đây là số lượng dữ liệu muốn đọc

I Q M DB T C

Ký hiệu các kích thước:

Ký hiệu	Tên	Số bit	Ví dụ
X	Bit	1 bit (mặc định nếu không chỉ định kích thước trong cú pháp, dùng cho loại dữ liệu boolean)	M0.3 (Merker byte 0, bit 3)
B	Byte	8 bit (Thường dùng cho số nguyên 0-255)	MB5 (Merker Byte 5)
W	Word	16 bit (Thường dùng cho số nguyên lớn)	MW10 (Merker Word tại byte 10)
D	DWord	32 bit (Thường dùng cho số thực)	MD20 (Merker DWord tại byte 20)

Với vùng DB thì cú pháp đặc biệt hơn:

```

Bit:    DB[n].DBX [byte] . [bit]
Byte:   DB[n].DBB [byte]
Word:   DB[n].DBW [byte]
DWord:  DB[n].DBD [byte]
```

Ví dụ:

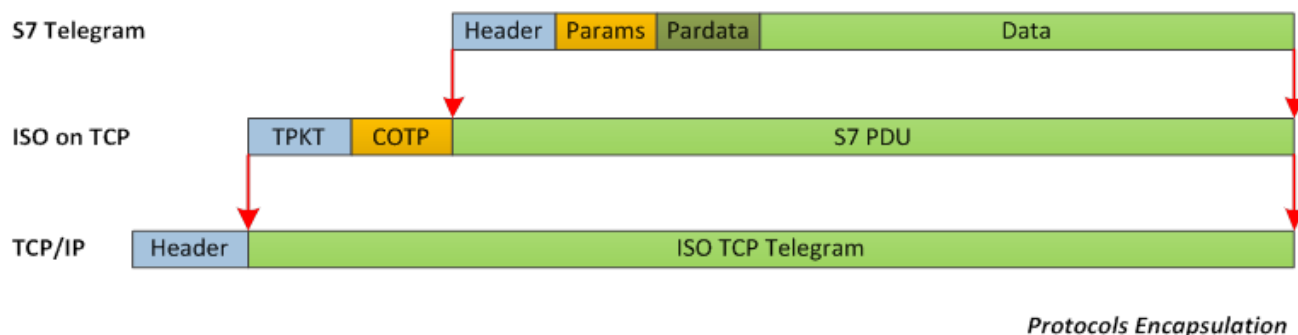
- DB1.DBX0.0 → bit 0 của byte 0 trong DB1
- DB1.DBX0.1 → bit 1 của byte 0 trong DB1

Siemens S7 sử dụng BIG-ENDIAN (byte có trọng số cao nhất nằm ở địa chỉ thấp nhất).

Các kiểu dữ liệu trong DB:

Type	Size	Ghi chú
BOOL	1 bit	True/False
BYTE	1 byte	0-255, unsigned
INT	2 bytes	-32768 đến 32767, signed
WORD	2 bytes	0-65535, unsigned
DINT	4 bytes	-2^31 đến 2^31-1
DWORD	4 bytes	0 đến 2^32-1
REAL	4 bytes	Float 32-bit (IEEE 754)
STRING	variable	[max_len][cur_len][chars...]
S5TIME	2 bytes	Thời gian Siemens (định dạng riêng)

Cấu trúc gói tin S7



Phần COTP và TPKT

Trong giao thức ISO-on-TCP mà S7 sử dụng, ISO cần giải quyết các khác biệt sau để tương thích với TCP:

1. TCP là giao thức hướng kết nối, trong khi ISO là giao thức không hướng kết nối (connectionless). Do đó, ISO cần sử dụng thêm **COTP (Connection-Oriented Transport Protocol)**. COTP chịu trách nhiệm thiết lập kết nối logic giữa hai thực thể đầu cuối trước khi dữ liệu S7 được trao đổi, giúp ISO-on-TCP có khả năng định tuyến qua các mạng từ xa.

Trong quá trình thiết lập kết nối COTP, tham số **TSAP (Transport Service Access Point)** đóng vai trò quan trọng trong việc định tuyến thông điệp đến đúng tiến trình bên trong CPU của PLC. Một TSAP thường gồm hai byte, có cấu trúc phản ánh loại giao tiếp và vị trí vật lý của CPU.

Thành phần TSAP	Giá trị	Ý nghĩa
Byte 1 (Loại thiết bị)	0x01, 0x02, 0x03	0x01 đại diện cho PG (Máy lập trình), 0x02 cho OP (Bảng vận hành), 0x03 cho các kết nối khác
Byte 2 (Vị trí)	Bits 0 - 4 cho Slot, Bits 5 - 7 cho Rack	Xác định vị trí của CPU trong tủ điện (ví dụ: Rack 0, Slot 2)

2. TCP là giao thức dạng luồng (stream-based), vì vậy **TPKT (ISO Transport Service on top of TCP)** được sử dụng làm lớp bao bọc để TCP có thể phân tách các gói tin có độ dài xác định.

Cụ thể, ngăn xếp giao thức (protocol stack) của Siemens sẽ tự động chèn một header TPKT trước dữ liệu COTP và S7. TPKT đóng vai trò tạo khung (framing). Header TPKT có độ dài 4 byte, trong đó:

- Byte đầu tiên của TPKT luôn là phiên bản (0x03)
- Byte thứ hai là phần dự phòng, luôn là 0x00
- Hai byte cuối cùng quy định tổng độ dài của gói tin (bao gồm cả header và payload). Khi PLC nhận được luồng byte từ TCP, nó sẽ đọc bốn byte này để xác định độ dài gói tin. Sau đó, PLC sẽ đợi đến khi nhận đủ số byte rồi mới chuyển khối dữ liệu lên tầng trên (S7comm) để xử lý.

Kết quả là ta có một giao thức vừa có độ dài thông điệp xác định (ưu điểm của ISO), vừa có thể truyền qua các bộ định tuyến mạng (ưu điểm của TCP).

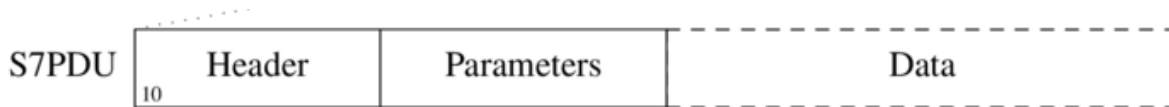
[!NOTE]

S7 hoạt động trên cổng TCP 102. COTP được định nghĩa trong RFC 905; TPKT được định nghĩa trong RFC 1006 và được cập nhật bởi RFC 2126.

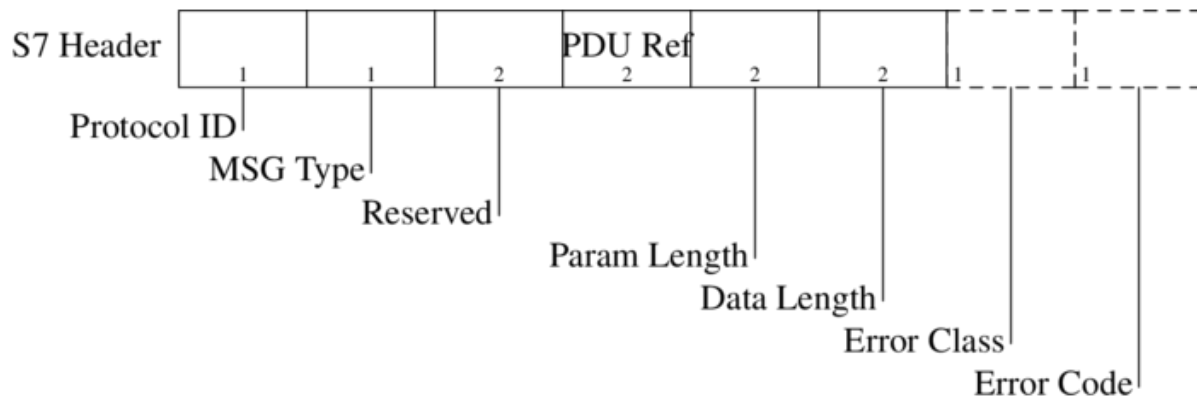
Phần S7 PDU

Giao thức S7 được thiết kế theo hướng chức năng (function-oriented) hoặc lệnh (command-oriented), tức là mỗi lần truyền sẽ bao gồm một lệnh điều khiển hoặc một phản hồi. Nếu một lệnh không vừa trong một PDU, nó sẽ được chia thành nhiều PDU liên tiếp.

Một lệnh bao gồm:



- Header:



Với:

- Protocol ID (kích thước 1 Byte): Luôn luôn là 0x32 để định danh giao thức S7comm.
- Message Type hoặc Remote Operating Service Control (ROSCTR) : Cho biết loại gói tin:

ROSCTR	Ý nghĩa
Job (1)	Yêu cầu từ Client tới Server như read/write memory, read/write blocks, start/stop device, setup communication
Ack (2)	Xác nhận nhận được yêu cầu từ Server, không kèm theo dữ liệu
Ack_Data (3)	Xác nhận nhận được yêu cầu từ Server, kèm theo dữ liệu trả về cho job request trước đó
User_Data (7)	Đây là phần mở rộng so với giao thức gốc, sử dụng cho các chức năng nâng cao. Nó chứa Function Group bao gồm các chức năng: Programmer commands, Cyclic data, Block functions, CPU functions, Security, Time functions. Trong các function này lại bao gồm nhiều subfunction khác. Ví dụ như trong nhóm function CPU Functions lại có: Read SZL, Message service, ...

- **Reserved** : Luôn là `0x0000` , không có ý nghĩa gì.
- **PDU Reference** : Một số do Client tạo ra để định danh yêu cầu. Khi PLC phản hồi, nó sẽ trả lại số này để Client biết phản hồi tương ứng với yêu cầu nào. Số tham chiếu này tăng dần theo từng yêu cầu mới được gửi đi.
- **Parameter length** : Độ dài của phần tham số trong gói tin, tính theo byte, biểu diễn kiểu Big-Endian.
- **Data length** : Độ dài của phần dữ liệu trong gói tin, tính theo byte, biểu diễn kiểu Big-Endian.
- **Error class** và **Error code** : Chỉ xuất hiện trong các gói tin phản hồi `ROSCTR = Ack_Data` , cho biết có lỗi gì xảy ra không.
- **Tập các tham số** (bao gồm tên và giá trị của các tham số, nếu có). Tùy theo loại lệnh (được xác định thông qua trường `S7 Header Function Code`), tập tham số trong phần này sẽ khác nhau. Các lệnh được chia thành các nhóm chức năng: Data Read/Write, Cyclic Data Read/Write, Directory Info, System Info, Block Move, PLC Control, Date and Time, Security và Programming.

Các bước thiết lập kết nối S7

5 8.766154	192.168.1.10	192.168.1.40	TCP	66 4305 → 102 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM	TCP Handshake
6 8.768131	192.168.1.40	192.168.1.10	TCP	60 102 → 4305 [SYN, ACK] Seq=0 Ack=1 Win=4096 Len=0 MSS=1460	
7 8.768198	192.168.1.10	192.168.1.40	TCP	54 4305 → 102 [ACK] Seq=1 Ack=1 Win=64240 Len=0	
8 8.768395	192.168.1.10	192.168.1.40	COTP	76 CR TPDU src-ref: 0x000f dst-ref: 0x0000	COTP
9 8.772175	192.168.1.40	192.168.1.10	COTP	76 CC TPDU src-ref: 0x0003 dst-ref: 0x000f	
10 8.772331	192.168.1.10	192.168.1.40	S7COMM	79 ROSCTR:[Job] Function:[Setup communication]	Init S7
11 8.776092	192.168.1.40	192.168.1.10	S7COMM	81 ROSCTR:[Ack_Data] Function:[Setup communication]	
12 8.776179	192.168.1.10	192.168.1.40	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]	S7 actual data exchange
13 8.776239	192.168.1.10	192.168.1.40	S7COMM	87 ROSCTR:[Userdata] Function:[Request] → [CPU functions] → [Read SZL] ID=0x0132 Index=0x0004	
14 8.782214	192.168.1.40	192.168.1.10	S7COMM	135 ROSCTR:[Userdata] Function:[Response] → [CPU functions] → [Read SZL] ID=0x0132 Index=0x0004	
15 8.782335	192.168.1.10	192.168.1.40	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]	
16 8.782713	192.168.1.10	192.168.1.40	S7COMM	87 ROSCTR:[Userdata] Function:[Request] → [CPU functions] → [Read SZL] ID=0x0000 Index=0x0000	
17 8.788255	192.168.1.40	192.168.1.10	S7COMM	301 ROSCTR:[Userdata] Function:[Response] → [CPU functions] → [Read SZL] (S7COMM fragment id=5)	
18 8.788347	192.168.1.10	192.168.1.40	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]	
19 8.788424	192.168.1.10	192.168.1.40	S7COMM	87 ROSCTR:[Userdata] Function:[Request] → [CPU functions] → [Read SZL]	
20 8.793186	192.168.1.40	192.168.1.10	S7COMM	105 ROSCTR:[Userdata] Function:[Response] → [CPU functions] → [Read SZL] (S7COMM reassembled id=5) ID=0x0000 Index=0x0000	

Tập *SampleCaptures/s7comm_reading_plc_status.pcap* từ <https://wiki.wireshark.org/S7comm>, với bộ lọc `tcp.port == 102` .

Thiết lập kết nối tới TCP 102 bằng Three-way Handshake của TCP

5 8.766154	192.168.1.10	192.168.1.40	TCP	66 4305 → 102 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM
6 8.768131	192.168.1.40	192.168.1.10	TCP	60 102 → 4305 [SYN, ACK] Seq=0 Ack=1 Win=4096 Len=0 MSS=1460
7 8.768198	192.168.1.10	192.168.1.40	TCP	54 4305 → 102 [ACK] Seq=1 Ack=1 Win=64240 Len=0
8 8.768395	192.168.1.10	192.168.1.40	COTP	76 CR TPDU src-ref: 0x000f dst-ref: 0x0000
9 8.772175	192.168.1.40	192.168.1.10	COTP	76 CC TPDU src-ref: 0x0003 dst-ref: 0x000f
10 8.772331	192.168.1.10	192.168.1.40	S7COMM	79 ROSCTR:[Job] Function:[Setup communication]
11 8.776092	192.168.1.40	192.168.1.10	S7COMM	81 ROSCTR:[Ack_Data] Function:[Setup communication]
12 8.776179	192.168.1.10	192.168.1.40	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]
13 8.776239	192.168.1.10	192.168.1.40	S7COMM	87 ROSCTR:[Userdata] Function:[Request] → [CPU functions] → [Read SZL] ID=0x0132 Index=0x0004
14 8.782214	192.168.1.40	192.168.1.10	S7COMM	135 ROSCTR:[Userdata] Function:[Response] → [CPU functions] → [Read SZL] ID=0x0132 Index=0x0004
15 8.782335	192.168.1.10	192.168.1.40	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]
16 8.782713	192.168.1.10	192.168.1.40	S7COMM	87 ROSCTR:[Userdata] Function:[Request] → [CPU functions] → [Read SZL] ID=0x0000 Index=0x0000
17 8.788255	192.168.1.40	192.168.1.10	S7COMM	301 ROSCTR:[Userdata] Function:[Response] → [CPU functions] → [Read SZL] (S7COMM fragment id=5)
18 8.788347	192.168.1.10	192.168.1.40	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]
19 8.788424	192.168.1.10	192.168.1.40	S7COMM	87 ROSCTR:[Userdata] Function:[Request] → [CPU functions] → [Read SZL]
20 8.793186	192.168.1.40	192.168.1.10	S7COMM	105 ROSCTR:[Userdata] Function:[Response] → [CPU functions] → [Read SZL] (S7COMM reassembled id=5) ID=0x0000 Index=0x0000

Three-way Handshake của TCP giữa `192.168.1.10` và PLC `192.168.1.40` ở gói tin 5, 6, 7

Thiết lập kết nối COTP

Sau khi kết nối TCP được thiết lập, Client sẽ gửi một yêu cầu kết nối **COTP CR** (COTP Connect Request) tới PLC. Yêu cầu này bao gồm thông tin về TSAP để định danh dịch vụ và vị trí CPU mà Client muốn giao tiếp.

8	8.768395	192.168.1.10	192.168.1.40	COTP	76	CR TPDU src-ref: 0x000f dst-ref: 0x0000
9	8.772175	192.168.1.40	192.168.1.10	COTP	76	CC TPDU src-ref: 0x0003 dst-ref: 0x000f
10	8.772331	192.168.1.10	192.168.1.40	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
11	8.776092	192.168.1.40	192.168.1.10	S7COMM	81	ROSCTR:[Ack_Data] Function:[Setup communication]
12	8.776179	192.168.1.10	192.168.1.40	COTP	61	DT TPDU (0) [COTP fragment, 0 bytes]
13	8.776239	192.168.1.10	192.168.1.40	S7COMM	87	ROSCTR:[Userdata] Function:[Request] -> [CPU func
14	8.782214	192.168.1.40	192.168.1.10	S7COMM	135	ROSCTR:[Userdata] Function:[Response] -> [CPU fun
15	8.782335	192.168.1.10	192.168.1.40	COTP	61	DT TPDU (0) [COTP fragment, 0 bytes]

> Frame 8: Packet, 76 bytes on wire (608 bits), 76 bytes captured (608 bits)	0000	00 1b 1b 23 eb
> Ethernet II, Src: ASUSTekCOMPU_84:5e:41 (90:e6:ba:84:5e:41), Dst: Siemens_23:eb:3b (00:1b:1b:23:eb:3b)	0010	00 3e 42 6c 40
> Internet Protocol Version 4, Src: 192.168.1.10, Dst: 192.168.1.40	0020	01 28 10 d1 00
> Transmission Control Protocol, Src Port: 4305, Dst Port: 102, Seq: 1, Ack: 1, Len: 22	0030	fa f0 83 b3 00
> TPKT, Version: 3, Length: 22	0040	00 c1 02 01 00

v ISO 8073/X.224 COTP Connection-Oriented Transport Protocol

Length: 17
 PDU Type: CR Connect Request (0x0e)
 Destination reference: 0x0000
 Source reference: 0x000f
 0000 = Class: 0
0. = Extended formats: False
0 = No explicit flow control: False
 Parameter code: src-tsap (0xc1)
 Parameter length: 2
 Source TSAP: 0100
 Parameter code: dst-tsap (0xc2)
 Parameter length: 2
 Destination TSAP: 0102
 Parameter code: tpdu-size (0xc0)
 Parameter length: 1
 TPDU size: 1024

Gói tin số 8 chứa TPKT ở ngăn xếp trên và yêu cầu COTP CR ở ngăn xếp dưới.

```
TPKT, Version: 3, Length: 22
  Version: 3
  Reserved: 0
  Length: 22
```

- **Version: 3** là phiên bản của TPKT, luôn có giá trị 3 theo tiêu chuẩn RFC 1006. Đây là giá trị cố định, cho biết gói tin tuân theo định dạng TPKT.
- **Length: 22** là tổng độ dài của gói tin TPKT, bao gồm header TPKT (4 byte) và payload COTP CR (18 byte). Khi nhận được gói tin này, PLC sẽ đọc 4 byte đầu tiên để xác định tổng độ dài là 22 byte. Sau đó, PLC sẽ đợi đến khi nhận đủ 22 byte rồi mới xử lý dữ liệu COTP CR bên dưới. Nếu gói tin bị cắt ngắn hoặc xảy ra lỗi trong quá trình truyền, PLC sẽ không nhận đủ số byte cần thiết và có thể bỏ qua gói tin hoặc gửi lại yêu cầu kết nối.

ISO 8073/X.224 COTP Connection-Oriented Transport Protocol

```
Length: 17
PDU Type: CR Connect Request (0x0e)
Destination reference: 0x0000
Source reference: 0x000f
0000 .... = Class: 0
.... ..0. = Extended formats: False
.... ...0 = No explicit flow control: False
Parameter code: src-tsap (0xc1)
Parameter length: 2
Source TSAP: 0100
Parameter code: dst-tsap (0xc2)
Parameter length: 2
Destination TSAP: 0102
Parameter code: tpdu-size (0xc0)
Parameter length: 1
TPDU size: 1024
```

Với:

- PDU Type: CR Connect Request (0x0e) cho biết đây là một yêu cầu kết nối COTP CR. Ngoài ra còn có CC Connect Confirm (0x0d) và DT Data Transfer (0x0f)
- Destination reference: 0x0000 và Source reference: 0x000f là các trường định danh mà COTP sử dụng để quản lý kết nối logic. Client tạo Source reference khi gửi yêu cầu kết nối. Ban đầu, Destination reference bằng 0 vì chưa có phiên kết nối. Nếu chấp nhận yêu cầu, PLC sẽ phản hồi bằng một gói COTP CC (Connect Confirm), trong đó Destination reference được gán giá trị Source reference mà Client đã gửi (0x000f), còn Source reference của PLC được gán một giá trị mới do PLC tạo ra. Hai bên sử dụng cặp giá trị này để quản lý phiên kết nối.
- Source TSAP: 0100 loại thiết bị kết nối tới PLC. Ở đây 0x01 đại diện cho PG (Máy lập trình).
- Destination TSAP: 0102 giao tiếp tới CPU nằm ở Rack 0, Slot 2 của PLC.
- TPDU size: 1024 là kích thước tối đa của TPDU mà Client có thể gửi (TPDU là gói tin chứa TPKT và PDU của S7comm).

Server phản hồi bằng một gói COTP CC (Connect Confirm) ở gói tin số 9.

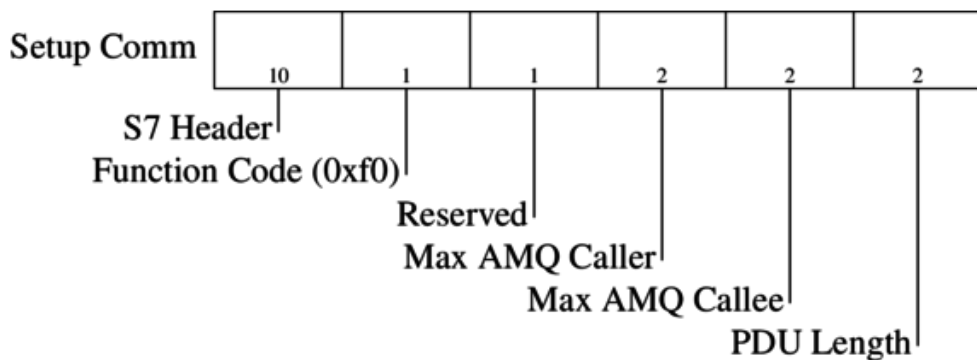
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol

Length: 17
PDU Type: CC Connect Confirm (0x0d)
Destination reference: 0x000f
Source reference: 0x0003
0000 = Class: 0
.... ..0. = Extended formats: False
.... ...0 = No explicit flow control: False
Parameter code: tpdu-size (0xc0)
Parameter length: 1
TPDU size: 1024
Parameter code: src-tsap (0xc1)
Parameter length: 2
Source TSAP: 0100
Parameter code: dst-tsap (0xc2)
Parameter length: 2
Destination TSAP: 0102

- Source reference: 0x0003 là giá trị PLC tạo ra để định danh nó trong session này. Client sẽ dùng giá trị này làm Destination reference trong các gói tin tiếp theo để gửi dữ liệu tới PLC.

Thiết lập kết nối S7comm ban đầu

Quá trình thiết lập ở tầng S7comm bắt đầu bằng việc gửi yêu cầu thiết lập giao tiếp S7 **Setup Communication** (gói tin số 10), bao gồm các tham số sau:



Trong đó:

- **S7 Header Function Code**: Cho biết loại lệnh S7, ở đây là 0xf0 tương ứng với lệnh thiết lập liên kết S7 (Setup Communication)
- **Max AmQ calling** quy định số yêu cầu mà Client có thể gửi đồng thời, còn **Max AmQ called** quy định số yêu cầu mà PLC có thể xử lý đồng thời. Hai bên sẽ đàm phán các tham số này khi thiết lập kết nối. Thông thường, cả hai đều bằng 1, nghĩa là Client chỉ có thể gửi yêu cầu mới sau khi nhận được phản hồi cho yêu cầu trước đó, còn PLC sẽ xử lý tuần tự từng yêu cầu.
- **PDU length**: Độ dài tối đa của PDU mà PLC có thể xử lý. Tham số này cũng được đàm phán lúc thiết lập kết nối.

Wireshark đã được tích hợp sẵn bộ phân tích gói tin S7comm, nên nó có thể giải mã trực tiếp các trường dữ liệu của S7comm trong phần Parameter và Data của gói tin.

S7 Communication

Header: (Job)

Protocol Id: 0x32

ROSCTR: Job (1)

Redundancy Identification (Reserved): 0x0000

Protocol Data Unit Reference: 512

Parameter length: 8

Data length: 0

Parameter: (Setup communication)

Function: Setup communication (0xf0)

Reserved: 0x00

Max AmQ (parallel jobs with ack) calling: 1

Max AmQ (parallel jobs with ack) called: 1

PDU length: 480

Với:

- Protocol Id: 0x32 là mã định danh cho giao thức S7comm.
- ROSCTR: Job (1) cho biết đây là một gói tin loại Job, tức là một yêu cầu từ Client tới Server.
- Protocol Data Unit Reference: 512 là một số tham chiếu do Client tạo ra để quản lý các yêu cầu. Client sẽ gán một số khác nhau cho mỗi yêu cầu gửi đi, và khi PLC phản hồi, nó sẽ trả lại số này để Client biết được phản hồi này tương ứng với yêu cầu nào.
- Parameter length: 8 và Data length: 0 cho biết phần tham số của gói tin dài 8 byte, trong khi phần dữ liệu dài 0 byte.
- Function: Setup communication (0xf0) cho biết đây là một yêu cầu thiết lập giao tiếp S7.
- Max AmQ (parallel jobs with ack) calling: 1 và Max AmQ (parallel jobs with ack) called: 1: cả hai đều bằng 1, nghĩa là Client chỉ có thể gửi yêu cầu mới sau khi nhận được phản hồi cho yêu cầu trước đó, còn PLC sẽ xử lý tuần tự từng yêu cầu.
- PDU length: 480 là kích thước tối đa, tính bằng byte, của payload trong một gói S7 mà Client mong muốn.

Nhận được gói tin, PLC phản hồi ở gói tin số 11:

S7 Communication

Header: (Ack_Data)

Protocol Id: 0x32

ROSCTR: Ack_Data (3)

Redundancy Identification (Reserved): 0x0000

Protocol Data Unit Reference: 512

Parameter length: 8

Data length: 0

Error class: No error (0x00)

Error code: 0x00

Parameter: (Setup communication)

Function: Setup communication (0xf0)

Reserved: 0x00

Max AmQ (parallel jobs with ack) calling: 1

Max AmQ (parallel jobs with ack) called: 1

PDU length: 240

Trong đó, Protocol Data Unit Reference: 512 tham chiếu đến mã định danh của gói tin yêu cầu trước đó; ROSCTR: Ack_Data (3) xác nhận rằng yêu cầu thiết lập giao tiếp đã được chấp nhận và không xảy ra lỗi (Error class: No error (0x00) , Error code: 0x00). PLC chấp nhận các thông số kết nối mà Client đề xuất, ngoại trừ PDU length được giảm xuống còn 240 byte.

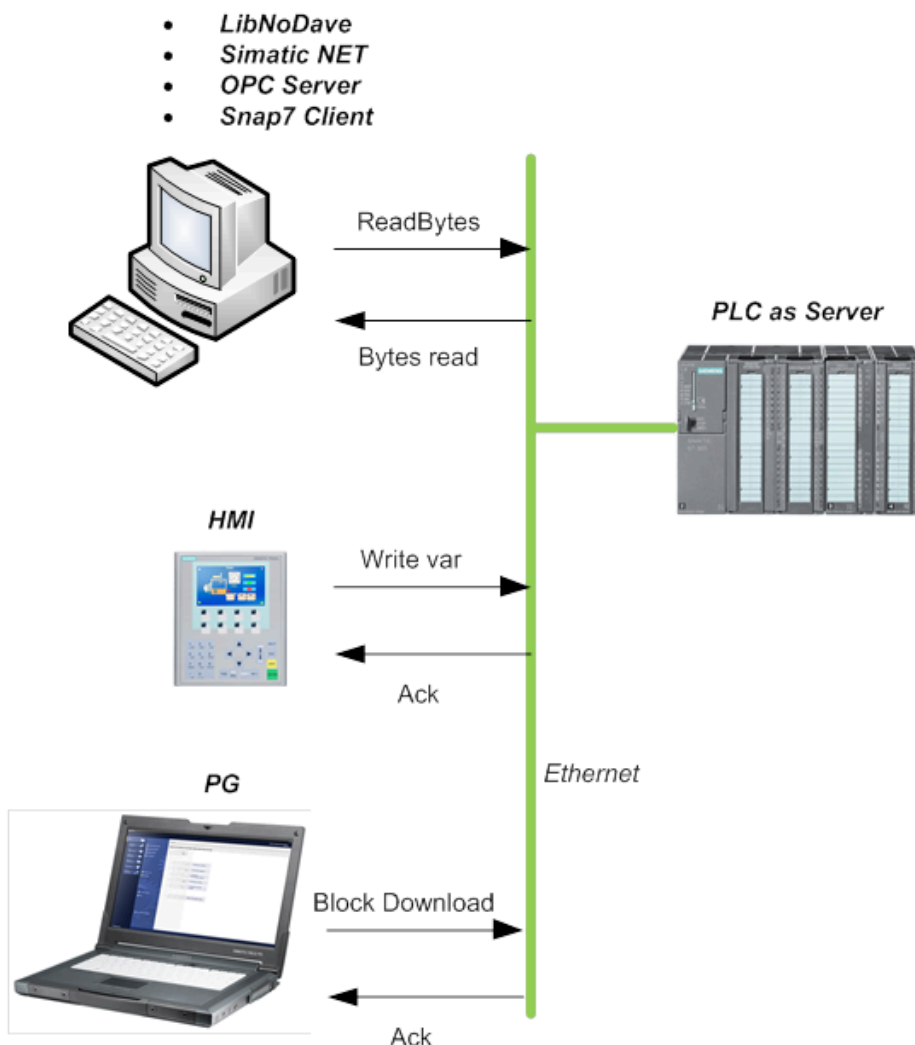
Trao đổi dữ liệu S7

Sau khi thiết lập xong kết nối S7, hai bên có thể bắt đầu trao đổi dữ liệu bằng các lệnh của S7 như Read/Write Var, ReadSZL, ... Tùy theo từng lệnh mà cấu trúc gói tin sẽ khác nhau.

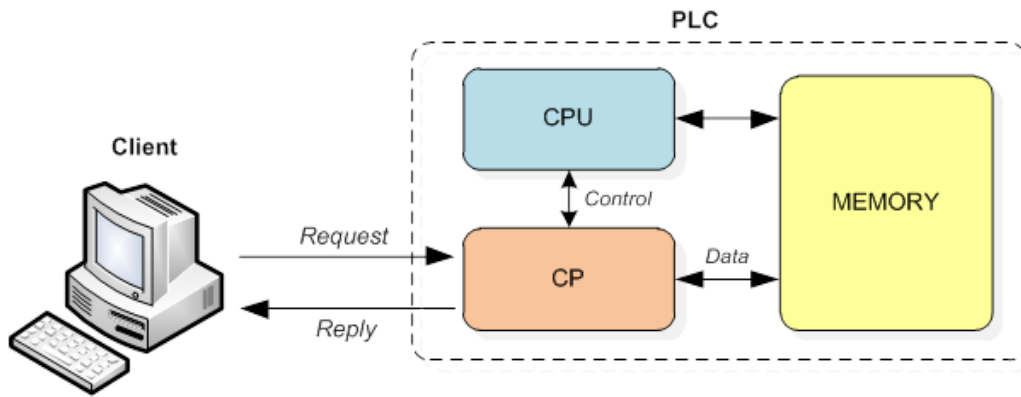
Các thành phần trong giao tiếp S7

Có ba tác nhân trong giao tiếp S7:

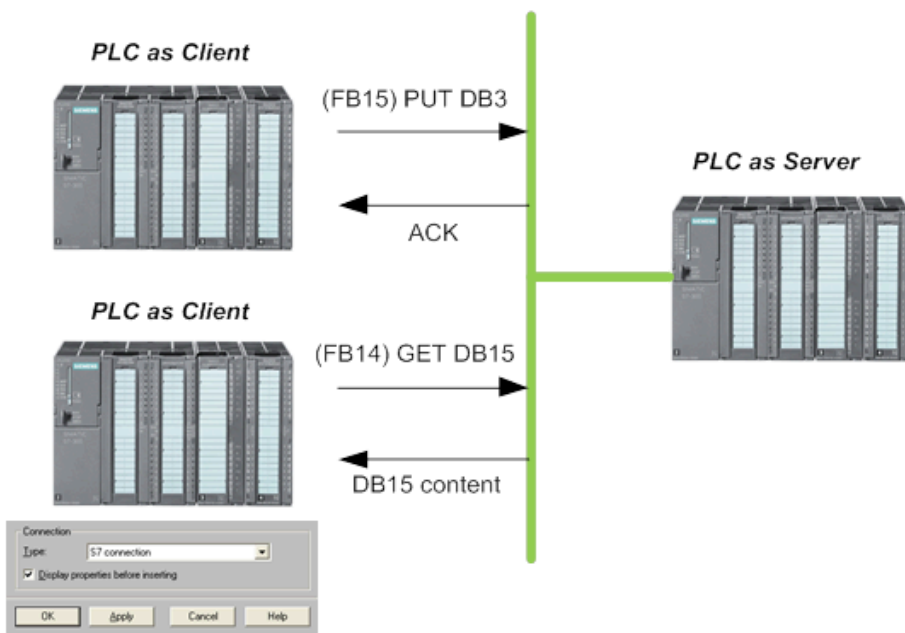
- **Client:** Thường là HMI, SCADA hoặc một máy trạm kỹ thuật (Engineering Station) có nhiệm vụ giám sát và điều khiển PLC. Client sẽ gửi các lệnh đọc/ghi dữ liệu, yêu cầu thực thi chương trình hoặc thay đổi cấu hình của PLC.
- **Server:** Đây chính là PLC. Nó nhận các yêu cầu từ Client, thực thi chúng và trả về kết quả. Server sẽ xử lý các lệnh như đọc/ghi dữ liệu, thực thi chương trình hoặc cung cấp thông tin về trạng thái của PLC. PLC giao tiếp thông qua **CP (Communication Processor)**, một mô-đun chuyên dụng để xử lý giao tiếp mạng. CP sẽ nhận các gói tin S7 từ mạng, giải mã và chuyển tiếp chúng đến CPU của PLC để thực thi. Sau khi CPU xử lý xong, kết quả sẽ được gửi qua CP để trả về cho Client.



Kiến trúc bên trong PLC Server



PLC cũng có thể đóng vai trò là Client khi cần giao tiếp với một PLC khác để lấy dữ liệu hoặc gửi lệnh điều khiển thông qua Get/Put:



- **Partner:** Tương tự kiến trúc peer-to-peer, sau khi kết nối được thiết lập, cả hai bên đều có thể gửi dữ liệu cho nhau mà không cần tuân theo mô hình request-reply nêu trên. Trong các tài liệu hướng dẫn của Siemens, thuật ngữ **Client-Client** được dùng cho khái niệm này. PLC gửi yêu cầu thiết lập kết nối được gọi là **Active Connection**, còn PLC nhận yêu cầu được gọi là **Passive Connection**. Sau đó, hai bên giao tiếp với nhau thông qua lệnh BSend/BRecv. Khi PLC A gọi BSend, PLC B cũng phải gọi BRecv cùng lúc để hoàn tất giao dịch.

